OXVERSE ACADEMY

Mobile App Development Curriculum

Professional Diploma · Cohort-based · Project-driven

DURATION	TOTAL WEEKS	LEVEL	PROJECTS
3 Months (12 Weeks)	12	Beginner to Industry Ready	12+

CAPSTONE
1 Published Cross-Platform App

PROGRAMME GOAL

Train learners to design, build, test, and publish production-grade cross-platform mobile applications using React Native, Expo, and TypeScript, covering native device APIs, backend integration, payments, animations, and real app store deployment.

OVERVIEW

A 12-week intensive mobile development diploma taking learners from JavaScript/TypeScript fundamentals through React Native and Expo architecture, navigation, state management, native device capabilities, offline data, authentication, backend and payment integration, advanced animations, testing, performance tuning, and finally publishing a real app to the Google Play Store and Apple App Store as a graded capstone.

No 82, Century Bus Stop, Ago Palace Way, Okota, Lagos · +234 814 846 2776

Course Outline

1. **Week 1:** TypeScript & Mobile Development Foundations
2. **Week 2:** React Native Core Components & Styling
3. **Week 3:** Navigation & App Architecture
4. **Week 4:** State Management & Data Fetching
5. **Week 5:** Native Device APIs
6. **Week 6:** Offline Storage & Data Persistence
7. **Week 7:** Authentication & Backend Integration
8. **Week 8:** Payments, Monetization & Third-Party Services
9. **Week 9:** Animations, Gestures & Advanced UI
10. **Week 10:** Testing, Debugging & Performance Optimization
11. **Week 11:** Native Modules, CI/CD & App Store Preparation
12. **Week 12:** Capstone: Build & Publish a Cross-Platform App

WEEK 1 OF 12

TypeScript & Mobile Development Foundations

Establish strong JavaScript/TypeScript fundamentals and set up the modern React Native + Expo development environment.

LEARNING OBJECTIVES

- Write type-safe TypeScript code using interfaces, types, and generics
- Set up a full React Native/Expo development environment on macOS/Windows/Linux
- Understand the mobile app architecture and how JS bridges to native code
- Run and debug apps on iOS simulator, Android emulator, and physical devices

DETAILED TOPIC BREAKDOWN

1.1 TypeScript Fundamentals for Mobile

- Primitive types, arrays, tuples, and enums
- Interfaces vs type aliases
- Union and intersection types
- Optional and readonly properties
- Generics in functions and interfaces
- Type narrowing and type guards
- Utility types: Partial, Pick, Omit, Record
- Strict mode and tsconfig.json configuration
- Async/await typing and Promise generics
- Modules, imports, and path aliases

1.2 React Native & Expo Ecosystem

- History and architecture of React Native (Bridge vs New Architecture/Fabric)
- What Expo is and Expo Managed vs Bare workflow
- Expo SDK overview and versioning
- Expo CLI vs React Native CLI
- EAS (Expo Application Services) overview
- JavaScript thread vs UI thread vs native modules
- Metro bundler basics
- Comparing React Native to Flutter and native development

1.3 Development Environment Setup

- Installing Node.js, Watchman, and package managers (npm/yarn/bun)
- Installing Xcode and iOS Simulator (macOS)
- Installing Android Studio, SDKs, and AVD emulator
- Creating a new Expo project with create-expo-app
- Configuring app.json/app.config.ts
- Running on physical devices with Expo Go
- Setting up VS Code with ESLint, Prettier, and React Native extensions
- Git repository setup and .gitignore for RN/Expo projects

1.4 Debugging & Dev Tools

- React Native Debugger and Flipper basics
- Console logging strategies and remote debugging
- Fast Refresh and hot reloading behavior
- Inspecting element trees with Dev Menu
- Common setup errors and troubleshooting (SDK mismatches, pod install issues)
- Using expo doctor to diagnose project health

HANDS-ON EXERCISES

- Convert 10 plain JavaScript functions into strictly typed TypeScript
- Build typed interfaces for a User, Product, and Order model
- Initialize 3 separate Expo projects with different templates
- Run the same app on iOS simulator, Android emulator, and Expo Go on a physical phone

ASSIGNMENTS

- Set up a fully working local dev environment and submit screenshots of the app running on all three targets
- Write a TypeScript utility library (10+ functions) with full type coverage and no 'any' usage

PROJECTS

- Scaffold and configure a 'Hello Mobile World' Expo TypeScript starter app with custom app icon, splash screen, and typed constants file

WEEKLY LEARNING OUTCOMES

- Comfortable writing idiomatic TypeScript
- Fully configured cross-platform mobile dev environment
- Understanding of RN/Expo architecture and tooling
- Ability to debug and troubleshoot setup issues independently

WEEKLY ASSESSMENT

Environment setup review + TypeScript coding test (20 typed exercises, timed)

WEEK 2 OF 12

React Native Core Components & Styling

Master the fundamental building blocks of React Native UIs and responsive styling systems.

LEARNING OBJECTIVES

- Build screens using core React Native components
- Apply Flexbox layout for responsive UI across screen sizes
- Style components using StyleSheet and design tokens
- Handle user input and touch interactions

DETAILED TOPIC BREAKDOWN

2.1 Core Components

View, Text, Image, ScrollView, SafeAreaView | TextInput and controlled inputs | TouchableOpacity, Pressable, and touch feedback | FlatList and SectionList for performant lists | Modal and Alert components | ActivityIndicator and loading states | KeyboardAvoidingView and keyboard handling | Platform-specific components with Platform.OS

2.2 Flexbox & Layout System

Flex direction, justify-content, align-items in RN's flexbox | Absolute vs relative positioning | Dimensions API and responsive units | useWindowDimensions hook for orientation changes | Percentage-based vs fixed layouts | Building responsive grids with FlatList numColumns | SafeAreaContext and notch/status bar handling | Aspect ratio and image scaling

2.3 Styling Systems

StyleSheet.create and style objects | Style composition and arrays | Theming with color palettes and design tokens | Dark mode with useColorScheme | Custom fonts with expo-font | NativeWind (Tailwind for React Native) setup and usage | Shadow and elevation cross-platform styling | Reusable styled component patterns

2.4 Forms & User Input

Controlled TextInput with validation | Building a login form UI | Handling keyboard types and input masking | Form libraries: React Hook Form for React Native | Displaying validation errors inline | Picker and Switch components | Date/time pickers with @react-native-community/datetimepicker

HANDS-ON EXERCISES

- Recreate 5 real app UI screens (Instagram profile, WhatsApp chat list, food delivery card) pixel-for-pixel
- Build a responsive grid layout that adapts to phone and tablet
- Implement dark mode toggle across an entire screen
- Build a multi-field signup form with client-side validation

ASSIGNMENTS

- Design and implement a reusable design system (Button, Card, Input, Badge components) with TypeScript props
- Build a fully responsive product listing screen using FlatList and custom styling

PROJECTS

- 'Social Feed UI Clone' - a static but fully responsive and styled social media feed screen with posts, likes, and comments UI

WEEKLY LEARNING OUTCOMES

- Fluency with all core React Native components
- Strong command of Flexbox layout for mobile
- A personal reusable component/design system library
- Ability to translate Figma designs into pixel-accurate RN UI

WEEKLY ASSESSMENT

UI clone challenge graded against a provided design mockup

WEEK 3 OF 12

Navigation & App Architecture

Implement multi-screen navigation patterns using React Navigation and structure scalable app architecture.

LEARNING OBJECTIVES

- Implement stack, tab, and drawer navigation
- Pass and type-check navigation parameters
- Structure a scalable folder architecture for mobile apps
- Handle deep linking and universal links

DETAILED TOPIC BREAKDOWN

3.1 React Navigation Fundamentals

Installing and configuring React Navigation with Expo | Native Stack Navigator setup and screen options | Bottom Tab Navigator with custom icons | Drawer Navigator for side menus | Nesting navigators (stack inside tabs) | Header customization and hiding headers | Screen transitions and animations config | Navigation lifecycle events (focus, blur)

3.2 Typed Navigation & Params

Defining ParamList types for stacks and tabs | Typing useNavigation and useRoute hooks | Passing and validating route parameters | Navigating programmatically (navigate, push, replace, goBack) | Resetting navigation stack on logout | Modal screens and presentation styles | Nested navigator type composition

3.3 App Architecture & Project Structure

Feature-based folder structure vs type-based structure | Separating screens, components, hooks, services, and utils | Environment variables with expo-constants and .env files | App-wide constants and theming configuration | Absolute imports and path aliases (tsconfig paths) | Creating a shared api/ service layer | Error boundary components for crash isolation

3.4 Deep Linking & App Flow

Configuring URL schemes with app.json | Handling deep links with expo-linking | Universal links for iOS and App Links for Android basics | Conditional navigation flows (onboarding vs authenticated vs guest) | Splash screen control with expo-splash-screen | Persisting and restoring navigation state

HANDS-ON EXERCISES

- Build a 4-tab app with nested stack navigators in each tab
- Implement typed navigation between 6 screens with parameter passing
- Create an onboarding flow that only shows on first app launch
- Add a working deep link that opens a specific product detail screen

ASSIGNMENTS

- Architect a full app skeleton with auth flow, tab navigation, and modal screens, fully typed with TypeScript
- Document your app's folder architecture and navigation map as a diagram

PROJECTS

- 'Multi-Screen Marketplace App Shell' - full navigation skeleton with onboarding, auth stack, tab navigation (Home, Search, Cart, Profile), and deep linking to product pages

WEEKLY LEARNING OUTCOMES

- Confident implementation of complex navigation trees
- Type-safe navigation across the entire app
- A scalable, maintainable project architecture
- Working deep linking implementation

WEEKLY ASSESSMENT

Architecture review + live navigation flow demo

WEEK 4 OF 12

State Management & Data Fetching

Manage local and global app state and fetch/cache remote data efficiently.

LEARNING OBJECTIVES

- Manage complex local state with hooks
- Implement global state management with Zustand/Redux Toolkit
- Fetch, cache, and synchronize server data with React Query
- Handle loading, error, and empty states gracefully

DETAILED TOPIC BREAKDOWN

4.1 React State & Hooks Deep Dive

useState, useEffect, useReducer patterns | useMemo and useCallback for performance | Custom hooks for reusable logic | Context API for shared state (theme, auth) | Avoiding prop drilling and re-render pitfalls | useRef for mutable values and imperative handles

4.2 Global State Management

Zustand store setup and slices | Redux Toolkit setup: store, slices, reducers | Async thunks for side effects | Selecting and memoizing state selectors | Persisting state with AsyncStorage middleware | Choosing Zustand vs Redux vs Context for a project | DevTools integration for state debugging

4.3 Data Fetching with React Query

Setting up TanStack Query in a React Native app | useQuery for GET requests and caching | useMutation for POST/PUT/DELETE operations | Query invalidation and refetching strategies | Optimistic updates | Pagination and infinite queries with useInfiniteQuery | Handling loading, error, and stale data states | Background refetching and network status awareness

4.4 API Integration Patterns

Structuring an Axios/fetch API client layer | Interceptors for auth tokens and error handling | Typing API responses with TypeScript | Environment-based API base URLs (dev/staging/prod) | Handling network errors and retries | Mocking APIs during development with MSW or json-server

HANDS-ON EXERCISES

- Build a Zustand store managing cart state across multiple screens
- Fetch a public REST API's data with React Query and render a paginated list
- Implement optimistic UI update for a 'like' button
- Add pull-to-refresh and infinite scroll to a FlatList

ASSIGNMENTS

- Build a complete product catalog feature: fetch, cache, filter, and paginate data from a real API
- Refactor a Context-based state solution into Zustand and justify the tradeoffs in a short writeup

PROJECTS

- 'E-Commerce Catalog & Cart' - product listing with infinite scroll, detail screen, cart managed globally, and persisted across app restarts

WEEKLY LEARNING OUTCOMES

- Mastery of local and global state management strategies
- Efficient data fetching and caching with React Query
- Robust handling of loading/error/empty UI states
- A working persisted shopping cart feature

WEEKLY ASSESSMENT

Code review of state architecture + live debugging challenge

WEEK 5 OF 12

Native Device APIs

Integrate native hardware and OS-level capabilities including camera, location, sensors, and permissions.

LEARNING OBJECTIVES

- Access camera and media library from the app
- Retrieve and use device location data
- Request and manage runtime permissions properly
- Use device sensors and haptic feedback

DETAILED TOPIC BREAKDOWN

5.1 Permissions Management

Expo Permissions API overview | Requesting camera, location, and notification permissions | Handling denied and blocked permission states | iOS Info.plist usage descriptions | Android manifest permissions configuration | Graceful permission re-request UX patterns

5.2 Camera & Media

expo-camera setup and live preview | Capturing photos and switching front/back camera | expo-image-picker for gallery selection | Image compression and resizing before upload | expo-av for video/audio playback and recording | expo-media-library for saving to device gallery | Building a QR/barcode scanner with expo-camera

5.3 Location & Maps

expo-location for foreground and background location | Getting current position and watching position changes | Reverse geocoding addresses from coordinates | react-native-maps setup and markers | Drawing routes and polylines on a map | Geofencing basics | Handling location permission edge cases (denied, low accuracy)

5.4 Sensors, Haptics & Notifications

expo-sensors: accelerometer, gyroscope, magnetometer | expo-haptics for tactile feedback | expo-notifications: local notifications setup | Push notification tokens and registration | Scheduling and canceling notifications | Handling notification taps and deep linking from notifications | Background tasks with expo-background-fetch

HANDS-ON EXERCISES

- Build a working QR code scanner that parses and displays scanned data
- Create a photo capture and gallery-save feature with compression
- Display the user's live location on a map with a custom marker
- Schedule a local notification that fires 10 seconds after a button press

ASSIGNMENTS

- Build a 'Check-In' feature combining camera capture, location tagging, and a map preview
- Implement a robust permissions flow with clear denied/blocked state UX for at least 3 permission types

PROJECTS

- 'Field Report App' - capture photo evidence, tag GPS location on a map, add notes, and receive a local notification confirming submission

WEEKLY LEARNING OUTCOMES

- Confident integration of camera, location, and sensor APIs
- Proper handling of runtime permissions across iOS and Android
- Working local push notification system
- A complete native-features-driven feature project

WEEKLY ASSESSMENT

Live demo of Field Report App on a physical device

WEEK 6 OF 12

Offline Storage & Data Persistence

Persist data locally, build offline-first experiences, and sync with remote sources.

LEARNING OBJECTIVES

- Store structured data locally using SQLite and AsyncStorage
- Build offline-first UX with sync queues
- Cache images and assets for offline use
- Secure sensitive data on-device

DETAILED TOPIC BREAKDOWN

6.1 AsyncStorage & Key-Value Storage

AsyncStorage API basics (get, set, remove, merge) | Storing and retrieving JSON objects | MMKV as a faster alternative to AsyncStorage | Building a typed storage wrapper/service | Persisting user preferences and app settings | Storage size limits and cleanup strategies

6.2 SQLite & Structured Local Data

expo-sqlite setup and database creation | Creating tables and running CRUD SQL queries | Drizzle ORM or WatermelonDB overview for RN | Migrations and schema versioning | Querying and filtering local data | Indexing for query performance

6.3 Offline-First Architecture

Detecting network status with NetInfo | Queueing mutations while offline | Syncing local changes to server on reconnect | Conflict resolution strategies (last-write-wins, merge) | Optimistic local writes with background sync | Caching API responses for offline viewing

6.4 Secure Storage & Asset Caching

expo-secure-store for sensitive tokens/credentials | Encrypting sensitive local data | expo-file-system for file read/write operations | Caching remote images with expo-image | Managing app cache size and clearing cache | Best practices: what never to store on-device

HANDS-ON EXERCISES

- Build a local SQLite-backed notes app with full CRUD
- Implement an offline network banner that queues actions when offline
- Store an auth token securely with expo-secure-store
- Cache a list of images for offline viewing

ASSIGNMENTS

- Convert the earlier E-Commerce Catalog project to work fully offline with cached product data and queued cart syncing
- Write a short technical doc comparing AsyncStorage, MMKV, and SQLite tradeoffs for different use cases

PROJECTS

- 'Offline Task Manager' - a fully offline-capable to-do/notes app with SQLite persistence, sync queue simulation, and secure storage for a mock auth token

WEEKLY LEARNING OUTCOMES

- Ability to design offline-first mobile experiences
- Comfort with SQLite and key-value local storage
- Secure handling of sensitive on-device data
- Working sync-queue pattern implementation

WEEKLY ASSESSMENT

Offline mode demo: disable network mid-demo and show app still functions

WEEK 7 OF 12

Authentication & Backend Integration

Implement production-grade authentication flows and connect the app to a real backend service.

LEARNING OBJECTIVES

- Implement email/password and social authentication
- Manage auth tokens, refresh flows, and protected routes
- Integrate a Backend-as-a-Service (Supabase/Firebase)
- Build and consume a REST/GraphQL API from the mobile app

DETAILED TOPIC BREAKDOWN

7.1 Authentication Fundamentals

JWT structure and access/refresh token flow | Secure token storage with expo-secure-store | Building login, signup, and forgot-password screens | Auth Context/store and protected navigation guards | Session persistence across app restarts | Handling token expiry and silent refresh | Biometric authentication with expo-local-authentication

7.2 Social & OAuth Login

Google Sign-In integration with expo-auth-session | Apple Sign-In (required for App Store apps with social login) | OAuth 2.0/PKCE flow fundamentals | Handling deep-link redirects during OAuth | Linking multiple auth providers to one account

7.3 Backend-as-a-Service Integration

Supabase project setup: Auth, Database, Storage | Firebase project setup: Auth, Firestore, Storage | Row-level security policies in Supabase | Realtime subscriptions for live data updates | File uploads to cloud storage buckets | Choosing between Supabase, Firebase, and a custom backend

7.4 Custom API Integration

Consuming a Node.js/Express REST API from React Native | GraphQL basics with Apollo Client in React Native | Handling CORS and mobile-specific network configs | Environment configuration for dev/staging/prod backends | Error handling and standardized API error responses | Rate limiting and request retry strategies

HANDS-ON EXERCISES

- Build a full login/signup/forgot-password flow with Supabase Auth
- Add Google Sign-In to an existing app
- Implement biometric login as a secondary auth method
- Set up realtime data sync for a chat-like feature

ASSIGNMENTS

- Build a complete authenticated app: signup, login, protected routes, profile editing, and logout, backed by Supabase or Firebase
- Write and connect a small custom Node.js API endpoint to the app for a feature of your choice

PROJECTS

- 'Social Habit Tracker' - authenticated app with email + Google login, user profiles stored in Supabase, and realtime updates when habits are marked complete

WEEKLY LEARNING OUTCOMES

- Production-ready authentication implementation
- Working social login integration
- Hands-on experience with a real backend-as-a-service
- Ability to connect to and consume any REST/GraphQL API

WEEKLY ASSESSMENT

Full auth flow demo including edge cases (expired token, wrong password, network failure)

WEEK 8 OF 12

Payments, Monetization & Third-Party Services

Integrate payment processing, in-app purchases, and common third-party SDKs used in production apps.

LEARNING OBJECTIVES

- Integrate a payment gateway for one-time and recurring payments
- Implement in-app purchases and subscriptions
- Add analytics and crash reporting
- Integrate maps, chat, or media SDKs as needed by app type

DETAILED TOPIC BREAKDOWN

8.1 Payment Gateway Integration

- Stripe React Native SDK setup
- Building a PaymentSheet checkout flow
- Handling payment success/failure/cancellation states
- Paystack/Flutterwave integration for African markets
- Webhooks and server-side payment verification (conceptual)
- PCI compliance basics and why you never store card data client-side

8.2 In-App Purchases & Subscriptions

- Apple In-App Purchase and Google Play Billing overview
- RevenueCat setup for cross-platform subscription management
- Configuring products and entitlements
- Restoring purchases and handling receipt validation
- Free trial and paywall UX patterns
- Testing IAP with sandbox accounts

8.3 Analytics & Monitoring

- Setting up Firebase Analytics or PostHog for mobile
- Tracking custom events (signup, purchase, screen_view)
- Sentry integration for crash and error reporting
- Performance monitoring basics
- Funnel analysis for onboarding and conversion
- Privacy-compliant analytics (App Tracking Transparency on iOS)

8.4 Common Third-Party SDKs

- Integrating a chat SDK (Stream Chat or similar) basics
- Embedding maps and geolocation-based search
- Social sharing with expo-sharing
- In-app reviews with expo-store-review
- Ad integration overview (AdMob) and monetization tradeoffs

HANDS-ON EXERCISES

- Implement a Stripe test-mode checkout for a mock product purchase
- Set up a paywall screen with RevenueCat sandbox subscription
- Add Sentry and trigger/capture a test crash
- Track 5 custom analytics events across an app flow

ASSIGNMENTS

- Add a complete monetization layer (one-time payment or subscription paywall) to a previous project app
- Set up crash reporting and analytics dashboards and submit a screenshot report of tracked events

PROJECTS

- 'Premium Fitness App' - free tier with limited workouts, paywall unlocking premium content via RevenueCat sandbox subscription, with analytics tracking and crash reporting wired in

WEEKLY LEARNING OUTCOMES

- Ability to integrate real payment and subscription systems
- Working analytics and crash reporting pipeline
- Understanding of app monetization strategies
- Experience wiring third-party SDKs into a React Native app

WEEKLY ASSESSMENT

Monetization flow demo + analytics dashboard review

WEEK 9 OF 12

Animations, Gestures & Advanced UI

Build fluid, native-feeling interactions using Reanimated and Gesture Handler.

LEARNING OBJECTIVES

- Build performant 60fps animations with Reanimated
- Implement gesture-driven interactions
- Create shared element and screen transitions
- Build advanced custom UI components

DETAILED TOPIC BREAKDOWN

9.1 React Native Reanimated Fundamentals

Shared values and useSharedValue | useAnimatedStyle and worklets | withTiming, withSpring, withDecay animation functions | Animating layout with entering/exiting animations | interpolate() for value mapping | Sequencing and chaining animations | Running animations on the UI thread vs JS thread

9.2 Gesture Handling

react-native-gesture-handler setup | Pan, tap, pinch, and long-press gestures | Combining gestures (simultaneous, exclusive, race) | Building swipe-to-delete list items | Draggable and sortable list interactions | Bottom sheet implementation with gestures | Pull-to-refresh custom gesture-based implementation

9.3 Screen & Shared Element Transitions

React Navigation screen transition customization | Shared element transitions between screens | Custom modal presentation animations | Parallax scroll effects | Skeleton loading screens and shimmer effects | Lottie animations with lottie-react-native

9.4 Advanced Custom Components

Building a custom animated tab bar | Building a custom animated bottom sheet from scratch | Building a swipeable card stack (Tinder-style) | Building a custom pull-to-refresh animation | Accessibility considerations for animated components

HANDS-ON EXERCISES

- Build a swipe-to-delete list with spring animation
- Create a Tinder-style swipeable card stack
- Build a custom animated bottom sheet component
- Add Lottie success/error animations to a form submission flow

ASSIGNMENTS

- Build a fully animated onboarding flow with parallax effects and progress indicators
- Recreate one advanced UI pattern from a popular app (Instagram stories bar, Spotify now-playing bar, or similar) with full gesture support

PROJECTS

- 'Interactive Discovery App' - a Tinder-style swipe card app (jobs, recipes, or products) with animated card stack, gesture-based like/dislike, and animated transitions between screens

WEEKLY LEARNING OUTCOMES

- Fluency with Reanimated worklets and shared values
- Ability to build complex gesture-driven interactions
- Portfolio-quality animated UI components
- Understanding of animation performance on mobile

WEEKLY ASSESSMENT

Live gesture/animation demo graded on smoothness and code quality

WEEK 10 OF 12

Testing, Debugging & Performance Optimization

Ensure app quality through automated testing, systematic debugging, and performance profiling.

LEARNING OBJECTIVES

- Write unit and component tests for React Native code
- Perform end-to-end testing of critical user flows
- Profile and optimize app performance
- Identify and fix memory leaks and unnecessary re-renders

DETAILED TOPIC BREAKDOWN

10.1 Unit & Component Testing

- Jest configuration for React Native/Expo
- Writing unit tests for utility functions and hooks
- React Native Testing Library setup
- Testing components: rendering, props, and user events
- Mocking API calls and native modules in tests
- Snapshot testing tradeoffs
- Test coverage reports and thresholds

10.2 End-to-End Testing

- Maestro setup for mobile E2E testing
- Writing E2E flows: login, checkout, navigation
- Detox as an alternative E2E framework overview
- Running E2E tests on CI
- Testing on multiple device sizes and OS versions

10.3 Performance Profiling

- Using React DevTools Profiler in React Native
- Identifying unnecessary re-renders with why-did-you-render
- FlatList performance tuning (getItemLayout, windowSize, removeClippedSubviews)
- Image optimization and lazy loading with expo-image
- Reducing bundle size and analyzing bundle with source-map-explorer
- Hermes engine and its performance benefits
- Memory leak detection and fixing common leak patterns (listeners, timers)

10.4 Debugging & Crash Resolution

- Reading and interpreting native crash logs (iOS/Android)
- Using Sentry breadcrumbs to trace crash causes
- Debugging native module linking issues
- Handling app freezes and ANRs (Application Not Responding)
- Systematic bug triage and reproduction workflow

HANDS-ON EXERCISES

- Write 15 unit tests covering a utility library and a custom hook
- Write component tests for a form with validation logic
- Create a Maestro E2E flow testing full login-to-checkout journey
- Profile a slow list screen and apply at least 3 optimizations

ASSIGNMENTS

- Achieve 70%+ test coverage on a previous project's core logic and submit the coverage report
- Take a deliberately unoptimized app screen, profile it, and write a before/after performance report

PROJECTS

- 'QA & Performance Audit' - take the E-Commerce Catalog project from Week 4, add a full unit/component test suite, one Maestro E2E flow, and a documented performance optimization pass

WEEKLY LEARNING OUTCOMES

- Confident writing unit, component, and E2E tests
- Ability to profile and systematically improve app performance
- Skill in reading crash logs and resolving native issues
- A demonstrable before/after performance case study

WEEKLY ASSESSMENT

Test suite review + performance report grading

WEEK 11 OF 12

Native Modules, CI/CD & App Store Preparation

Extend apps with custom native code when needed, automate builds, and prepare all assets required for store submission.

LEARNING OBJECTIVES

- Understand when and how to write custom native modules
- Configure EAS Build for automated iOS/Android builds
- Set up CI/CD pipelines for mobile apps
- Prepare all app store assets, metadata, and compliance requirements

DETAILED TOPIC BREAKDOWN

11.1 Native Modules & Config Plugins

When to eject/prebuild vs stay in Expo managed workflow | Writing a simple native module (Swift/Kotlin) overview | Using Expo Config Plugins to modify native projects | Installing and linking third-party native libraries | Handling native dependency conflicts | expo prebuild and continuous native generation (CNG)

11.2 EAS Build & Automated Builds

EAS CLI setup and eas.json configuration | Build profiles: development, preview, production | Configuring iOS credentials (certificates, provisioning profiles) | Configuring Android credentials (keystore management) | Triggering builds and monitoring build logs | Internal distribution builds for beta testers | EAS Update for over-the-air JS updates

11.3 CI/CD for Mobile

GitHub Actions workflow for automated testing on PRs | Automating EAS builds from CI on merge to main | Automated version and build number bumping | Fastlane overview for release automation | Branch strategies for mobile release cycles (main/staging/release) | Automated changelog generation

11.4 App Store Asset Preparation

App icon and splash screen requirements per platform | Screenshot generation for multiple device sizes | Writing an effective app store listing (title, keywords, description) | Privacy policy and terms of service requirements | Apple App Privacy 'nutrition label' declarations | Google Play Data Safety form requirements | Age ratings and content guidelines compliance

HANDS-ON EXERCISES

- Configure eas.json with development, preview, and production profiles
- Trigger an EAS internal distribution build and install it on a real device
- Set up a GitHub Actions workflow that runs tests on every pull request
- Generate app store screenshots for 3 different device sizes

ASSIGNMENTS

- Fully configure EAS Build and produce a working preview build installable via QR code
- Prepare a complete app store listing package: icon, screenshots, description, privacy policy draft, and Data Safety form answers

PROJECTS

- 'Release Readiness Package' - take your capstone-in-progress app and produce a signed preview build plus complete store listing assets ready for submission

WEEKLY LEARNING OUTCOMES

- Ability to configure and trigger production mobile builds
- Working CI pipeline for a mobile codebase
- Complete, compliant app store asset package
- Understanding of native module integration boundaries

WEEKLY ASSESSMENT

Build pipeline review + store asset package audit

WEEK 12 OF 12

Capstone: Build & Publish a Cross-Platform App

Plan, build, polish, and submit a complete original mobile application to the Apple App Store and/or Google Play Store.

LEARNING OBJECTIVES

- Independently scope and plan a production mobile app
- Apply all prior skills to build a polished, tested, monetized app
- Submit a real app for review to Apple App Store and/or Google Play Store
- Present and defend technical and product decisions

DETAILED TOPIC BREAKDOWN

12.1 Capstone Planning & Scoping

- Choosing a viable app idea and defining core MVP scope
- User flow mapping and wireframing
- Technical architecture planning document
- Choosing backend (Supabase/Firebase/custom API)
- Feature prioritization: must-have vs nice-to-have
- Setting a realistic build timeline across the week

12.2 Full Build Sprint

- Implementing navigation, auth, and core screens
- Integrating chosen backend and data layer
- Adding at least one native device feature
- Implementing offline support where relevant
- Adding animations and polish pass
- Writing tests for critical flows
- Adding analytics and crash reporting

12.3 QA, Polish & Compliance

- Full regression testing across iOS and Android
- Accessibility pass (labels, contrast, touch targets)
- Performance and bundle size final check
- Preparing privacy policy and legal compliance
- Final app icon, splash screen, and branding polish
- Edge case handling (no network, empty states, errors)

12.4 Submission & Launch

- Creating App Store Connect and Google Play Console listings
- Submitting production build via EAS Submit
- Responding to reviewer feedback and rejections
- Setting up post-launch monitoring dashboards
- Planning a v1.1 roadmap based on projected user feedback
- Writing release notes and marketing copy

12.5 Presentation & Portfolio

- Recording a polished app demo video
- Writing a case study for the portfolio (problem, process, outcome)
- Preparing a technical walkthrough presentation
- Publishing project on GitHub with a professional README
- Presenting to peers and instructors for final review

HANDS-ON EXERCISES

- Draft and get sign-off on a one-page capstone scope document
- Build and demo a working alpha version by mid-week

- Run a full manual QA pass using a written test checklist
- Record a 2-minute demo video of the finished app

ASSIGNMENTS

- Submit the completed app build (EAS build link/APK/TestFlight link) along with source code repository
- Submit a written case study document covering architecture, challenges, and outcomes

PROJECTS

- Capstone: an original, fully-featured cross-platform mobile app including authentication, backend integration, at least one native device feature, offline support, animations, tests, and a real submission to the Apple App Store and/or Google Play Store

WEEKLY LEARNING OUTCOMES

- A published (or submission-ready) real-world mobile app
- A complete professional portfolio case study
- Demonstrated end-to-end mobile development competency
- Confidence presenting and defending technical work publicly

WEEKLY ASSESSMENT

Final capstone demo day presentation, code review, and store submission verification